

3.8 Cascading Types and Transactional in Relational Queries

• Why Cascading and @Transactional Matter Together:

- In JPA, entities are often related:
 - Parent <---> ChildAggregate
 - roots <---> dependent entities
- Two questions arise:
 - What should happen to related entities when I save/update/delete one entity? → **CascadeWhen**
 - should DB operations be executed atomically and consistently? → **@Transactional**

✓ Cascade controls what operations propagate

✓ @Transactional controls when they are committed or rolled back

• Cascading Types:

- **Definition:** Cascade types define whether operations on a parent entity should be automatically applied to its related child entities.

- **CascadeType.PERSIST :**

- ✓ Propagates save / persist

```
Order order = new Order();
OrderItem item = new OrderItem();
order.getItems().add(item);
entityManager.persist(order);
```

- Result:

Order → INSERT

OrderItem → INSERT (automatically)

- Without cascade → child not saved

- **CascadeType.MERGE :**

- ✓ Propagates update / merge

```
Order detachedOrder = getDetachedOrder();
entityManager.merge(detachedOrder);
```

- Result:

- Order updated

- Child entities merged automatically

- Used heavily in **detached entity reattachment**

- **CascadeType.REMOVE :**

- ✓ Propagates delete

```
entityManager.remove(order);
```

- Result:

- Parent and children refreshed from DB

- Rarely used

3.8 Cascading Types and Transactional in Relational Queries

- **CascadeType.REFRESH :**

- ✓ Reloads entity state from DB
`entityManager.refresh(order);`
- Result:
 - Parent and children refreshed from DB
- Rarely used

- **CascadeType.DETACH :**

- ✓ Detaches child entities
`entityManager.detach(order);`
- Result:
 - Order detached
 - Children detached

- **CascadeType.ALL :**

- ✓ Applies ALL cascade operations
- Equivalent to: **PERSIST + MERGE + REMOVE + REFRESH + DETACH**

- **What cascade does (alone):**

```
@OneToMany(cascade = CascadeType.ALL)
private List<OrderItem> items;
```

- **This only means:**

- When Order is persisted → persist OrderItem
- When Order is removed → remove OrderItem
- When Order is merged → merge OrderItem

- **It does NOT:**

- open a transaction
- commit anything
- guarantee DB execution

- It only defines propagation rules.

- **What @Transactional does:**

```
@Transactional
public void placeOrder(Order order) {
    orderRepository.save(order);
}
```

- **Spring does now:**

1. Opens a transaction
2. Creates a persistence context
3. Hibernate schedules:
 - INSERT order
 - INSERT order_items (**because of cascade**)
4. On method end → **flush + commit**
5. If exception → rollback everything

- **Without @Transactional:**

- Persistence context may close early
- Flush timing is unpredictable
- Cascade operations may not reach DB
- Lazy loading may fail
- Partial operations may occur

3.8 Cascading Types and Transactional in Relational Queries

- Example showing both:

```
@Service
public class OrderService {
    @Transactional
    public void createOrder(Order order) {
        orderRepository.save(order);
        // cascade persists items
        // transaction commits all together
    }
}
```

- **Outcome:**
 - Either Order + all Items **saved**
 - Or nothing saved (**rollback**)
- That's the real power.
- Cascade defines which entity operations propagate, while @Transactional ensures those operations are executed atomically within a single transaction.
- Cascade without @Transactional is unsafe.
- @Transactional without correct cascade is incomplete.